

Lab: Using Google Analytics in Flutter Apps

Estimated time: **30 minutes**



Welcome to this hands-on lab where you will learn how to integrate Google Analytics into your Flutter app using Firebase Analytics. You will explore key features like tracking user interactions, monitoring real-time data, and analyzing user behavior to make data-driven decisions that enhance your app's performance.

Objectives

After completing this lab, you will be able to:

- Set up a basic Flutter application
- Install and configure Firebase Analytics
- Instrument your app to log different types of events

- View and analyze these events in the Firebase console

About Cloud IDE

Running the lab

This lab is designed to be completed using a Cloud IDE environment. You will be writing and executing Dart code directly within the IDE.

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands-on labs for course and project-related labs. It is an open-source IDE (Integrated Development Environment).

Important notice about this lab environment

Please be aware that sessions for this lab environment are not persisted. Every time you connect to this lab, a new environment is created for you. Any data you may have saved in the earlier session would get lost. Plan to complete these labs in a single session, to avoid losing your data.

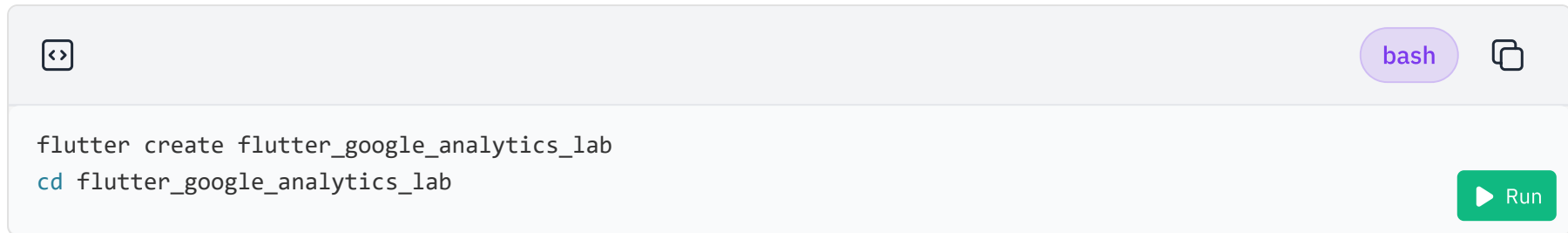
Key terms

- Google Analytics: A web analytics service offered by Google that tracks and reports website traffic and user behavior.
- Firebase Analytics: A free app measurement solution that provides insights on app usage and user engagement.
- Conversion tracking: Measures how well your app achieves specific goals, such as sign-ups or purchases.
- Real-time data: Immediate data about user interactions as they happen.

Step 1: Prerequisites

Before starting this lab, ensure you have the following setup in your Cloud IDE:

1. Create a new Flutter project:

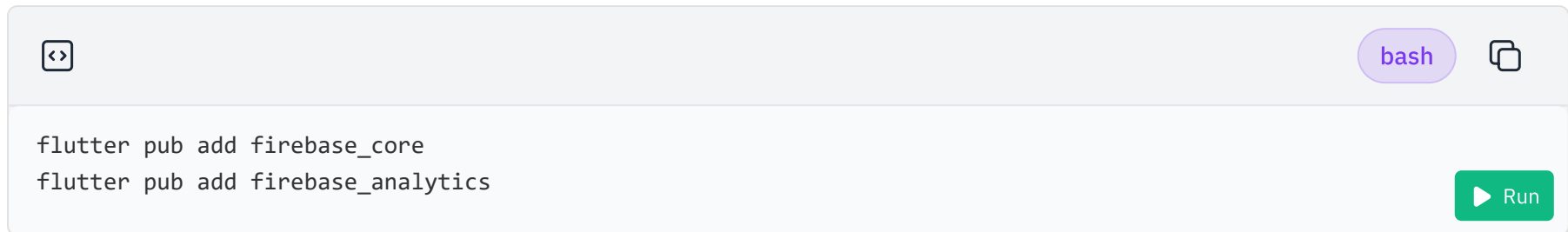
A screenshot of a Cloud IDE terminal window. The terminal has a light gray header bar with a code icon on the left, a purple 'bash' button in the center, and a copy icon on the right. The main area is white and contains two lines of code: 'flutter create flutter_google_analytics_lab' and 'cd flutter_google_analytics_lab'. A green 'Run' button with a play icon is located at the bottom right of the terminal area.

```
flutter create flutter_google_analytics_lab
cd flutter_google_analytics_lab
```

Step 2: Install Firebase Analytics

Open the terminal and cd into your directory if you closed the terminal since the previous step. Run the following command in the terminal to add the dependencies.

1. Add the Firebase Core and Firebase Analytics libraries to your project by running:



A terminal window with a light gray header bar. On the left is a code icon (two arrows pointing left and right). On the right is a purple pill-shaped button with the text 'bash' and a copy icon. The main area of the terminal is white and contains two lines of text: 'flutter pub add firebase_core' and 'flutter pub add firebase_analytics'. In the bottom right corner of the terminal area is a green button with a white play icon and the text 'Run'.

```
flutter pub add firebase_core
flutter pub add firebase_analytics
```

2. Ensure your `pubspec.yaml` file has been updated successfully with the Firebase Analytics dependency.



yaml



```
dependencies:
```

```
  flutter:
```

```
    sdk: flutter
```

```
# The following adds the Cupertino Icons font to your application.
```

```
# Use with the CupertinoIcons class for iOS style icons.
```

```
  cupertino_icons: ^1.0.8
```

```
  firebase_core: ^3.8.1
```

```
  firebase_analytics: ^11.3.6
```

Step 3: Instrument your app to log events

You are given a very simple app with two buttons, **Log Button 1 Event** and **Log Button 2 Event**. The app is purposefully simple so that it's easier to illustrate the main idea of using Google Analytics without any distraction.

Firebase Analytics Demo

DEBUG

Log Button 1 Event

Log Button 2 Event

1. Open your `lib/main.dart` file in the cloud IDE and import the Firebase Analytics package at the top of your file along with other files:



dart



```
import 'package:firebase_analytics/firebase_analytics.dart';  
import 'package:flutter/material.dart';  
import 'package:firebase_core/firebase_core.dart';  
import 'firebase_options.dart';  
import 'package:firebase_analytics/firebase_analytics.dart';
```

Let's discuss the `firebase_options.dart` file in the next few steps.

2. Configure firebase library in `main` method:



dart



```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  runApp(MyApp());  
}
```

3. Initialize Firebase Analytics in the `build` method of the `MyApp` class:

 dart 

```
class MyApp extends StatelessWidget {  
  // Access the FirebaseAnalytics instance  
  static final FirebaseAnalytics analytics = FirebaseAnalytics.instance;  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Firebase Analytics Demo',  
      home: MyHomePage(analytics: analytics),  
    );  
  }  
}
```

4. Add buttons to your app that log different types of events when clicked. For example, add the first button as follows:

 dart 

```
ElevatedButton(  
  onPressed: _logButton1Event,  
  child: Text('Log Button 1 Event'),  
),
```

Next, add the `logEvent` method to post events to Google Analytics:



dart



```
void _logButton1Event() {  
  widget.analytics.logEvent(  
    name: 'button_1_clicked',  
    parameters: {'button_name': 'Button 1'},  
  );  
  print('Button 1 clicked event logged');  
}
```

Do the same for the second button. The next step gives you the complete code if you are stuck. This is your chance to test your knowledge, so give it a try before moving on!

Step 4: Final code for the application

The previous step asked you to add code for the buttons and attach logging methods. Here is the complete code if you got stuck and need some help:



python



```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';
import 'package:firebase_analytics/firebase_analytics.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  // Access the FirebaseAnalytics instance
  static final FirebaseAnalytics analytics = FirebaseAnalytics.instance;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Firebase Analytics Demo',
      home: MyHomePage(analytics: analytics),
    );
  }
}

class MyHomePage extends StatefulWidget {
  final FirebaseAnalytics analytics;

  MyHomePage({Key? key, required this.analytics}) : super(key: key);

  @override
```

```

class _MyHomePageState extends State<MyHomePage> {
  void _logButton1Event() {
    widget.analytics.logEvent(
      name: 'button_1_clicked',
      parameters: {'button_name': 'Button 1'},
    );
    print('Button 1 clicked event logged');
  }

  void _logButton2Event() {
    widget.analytics.logEvent(
      name: 'button_2_clicked',
      parameters: {'button_name': 'Button 2'},
    );
    print('Button 2 clicked event logged');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Firebase Analytics Demo'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            ElevatedButton(
              onPressed: _logButton1Event,
              child: Text('Log Button 1 Event'),
            ),
            SizedBox(height: 20), // Adds space between the two buttons
            ElevatedButton(
              onPressed: _logButton2Event,
              child: Text('Log Button 2 Event'),
            ),
          ],
        ),
      ),
    );
  }
}

```

```
    },  
    },  
);  
}  
}
```


Step 5: Create Firebase project and configuration file

1. Go to the [Firebase Console](#) and create a new project. You will use this project when creating Flutter configuration file for your application.
2. The next step is to generate Flutter configuration in the cloud IDE. First, install the firebase tools:

 bash 

```
npm install firebase-tools
```

 Run

The output should look like this.

 plaintext 

```
changed 642 packages, and audited 642 packages in 2m

69 packages are looking for funding
  run `npm fund` for details

4 high severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

3. Ensure you are in the directory of your application. Run the following command to add firebase library to your terminal path

 bash 

```
export PATH=$PATH:/home/project/flutter_google_analytics_lab/node_modules/.bin
```

 Run

4. Next, log into firebase:

 bash 

```
firebase login --no-localhost
```

 Run

You will be moved through a wizard in the terminal to log into your firebase account. Copy the URL and open it in a new browser tab. Copy the authentication code that is generated and paste it at the prompt.



plaintext



```
i  Firebase optionally collects CLI and Emulator Suite usage and error reporting information to help improve our  
  
? Allow Firebase to collect CLI and Emulator Suite usage and error reporting information? Yes  
i  To change your data collection preference at any time, run `firebase logout` and log in again.
```

To sign in to the Firebase CLI:

1. Take note of your session ID:

```
59524
```

2. Visit the URL below on any device and follow the instructions to get your code:

```
https://auth.firebase.tools/login?code_challenge=BnpXLTjSOqofp3WPICgU_KVn5341h17zyRDa06VRfTI&session=595248ad
```

3. Paste or enter the authorization code below once you have it:

```
? Enter authorization code:
```

You will see a message confirming successful login.

4. Install the FlutterFire CLI by running the following command:



bash



```
dart pub global activate flutterfire_cli
```

▶ Run

bash

```
export PATH="$PATH": "$HOME/.pub-cache/bin"
```

 Run

5. Finally, run the following command from your application folder to generate the configuration file:

bash

```
flutterfire configure
```

 Run

6. This will start a multi-step wizard in the terminal. First, select the project from the list of available projects:

```
i Found 3 Firebase projects.
? Select a Firebase project to configure your Flutter application with >
  my-first-firebase-projec-2077e (My First Firebase Project)
  sample-476b3 (sample)
> test-30cd3 (test)
  <create a new project>
```

Next, pick the platform for which you are building. Use the arrow keys and space bars to select or deselct. You just need web to be checked.

```
? Which platforms should your configuration support (use arrow keys & space to select)? >
  android
  ios
  macos
✓ web
  windows
```

The complete output will look like this at the end:

 ruby 

```
$ flutterfire configure
? Generated FirebaseOptions file /dart-flutter/temp/google_an
✓ Generated FirebaseOptions file /dart-flutter/temp/google_analytics_app/lib/firebase_options.dart already exist
i Found 3 Firebase projects.
✓ Select a Firebase project to configure your Flutter application with · test-30cd3 (test)
✓ Which platforms should your configuration support (use arrow keys & space to select)? · web
i Firebase web app google_analytics_app (web) registered.

Firebase configuration file lib/firebase_options.dart generated successfully with the following Firebase apps:

Platform  Firebase App Id
web       1-126844110936-web-6142d0d09ef5d07ba8222d

Learn more about using this file and next steps from the documentation:
> https://firebase.google.com/docs/flutter/setup
```

This should generate `lib/firebase_options.dart`. This file is being used in `main.dart` as posted in the previous step:



dart



```
import 'firebase_options.dart';  
...  
await Firebase.initializeApp(  
  options: DefaultFirebaseOptions.currentPlatform,  
);
```

Step 6: Run the flutter app

1. Change to the home directory

 bash 

```
cd /home/project
```

 Run

2. Set the `PROJECT_DIR` variable. This should be set to the `lib` directory of your Flutter app folder.

 bash 

```
export PROJECT_DIR=/home/project/flutter_google_analytics_lab/lib
```

 Run

3. Get the script and save to the `/home/project` folder.

 bash 

```
wget -O /home/project/hot_reload.sh https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/S3cw83zxb>
```

 Run

4. Make the script executable by using the `chmod` command.

bash

```
chmod +x ./hot_reload.sh
```

Run

5. Run the command.

bash

```
./hot_reload.sh
```

Run

You should see output like this:

plaintext

```
theia@theia-captainfedo1:/home/project$ ./hot_reload.sh

Using PROJECT_DIR: /home/project/flutter_google_analytics_lab/lib
inotify-tools is already installed.
lsof is already installed.
Starting Flutter web server...
Port 8080 is free.

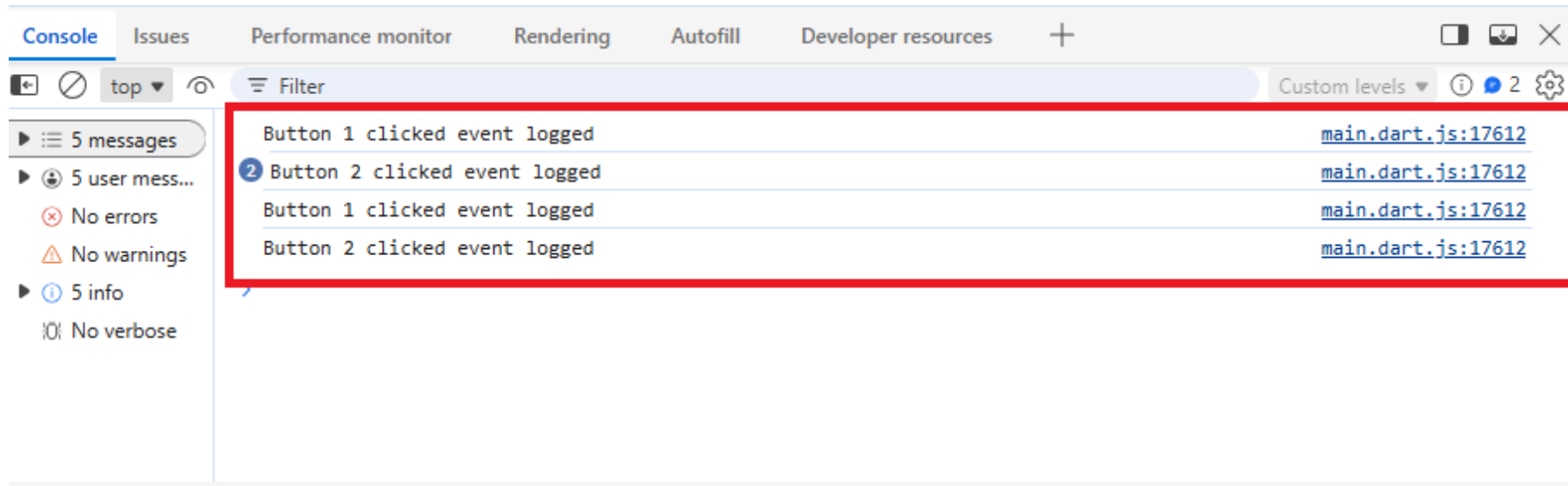
Compiling lib/main.dart for the Web...                               984ms
✓ Built build/web
Flutter server started with PID 17675
Setting up watches. Beware: since -r was given, this may take a while!
Watches established.
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```


- Open the **Skills Network Toolbox** in the left pane of the Cloud IDE.
- Click the **Launch Application** button.
- Enter **8080** as the port.
- Select the **Launch in new window** command to open the applicaiton in a new tab.

Step 7: Implement conversion tracking and analyze data

1. Select the tab running the application. Click the buttons randomly to send events to Google Analytics.

To see the events, open your app in the browser, right-click anywhere on the page, click “Inspect”, and then open the “Console” tab. The events will be displayed there when you interact with the app.



2. Use real-time data to monitor user interactions. Navigate to the "Realtime Analytics" section in the Firebase Console to view how users interact with your app live.

The screenshot shows the Firebase console interface. At the top, the 'newproject' dropdown is highlighted with a red box. On the left sidebar, 'Realtime Analytics' is highlighted with a red box. The main content area displays the 'Realtime overview' page. The 'Event count by Event name' chart is highlighted with a red box, showing a bar chart for 'button_2_clicked' with a count of 49 (53.85%). Below the chart is a table of event counts.

EVENT NAME	EVENT COUNT
button_2_clicked	49
button_1_clicked	34
page_view	5
user_engagement	3

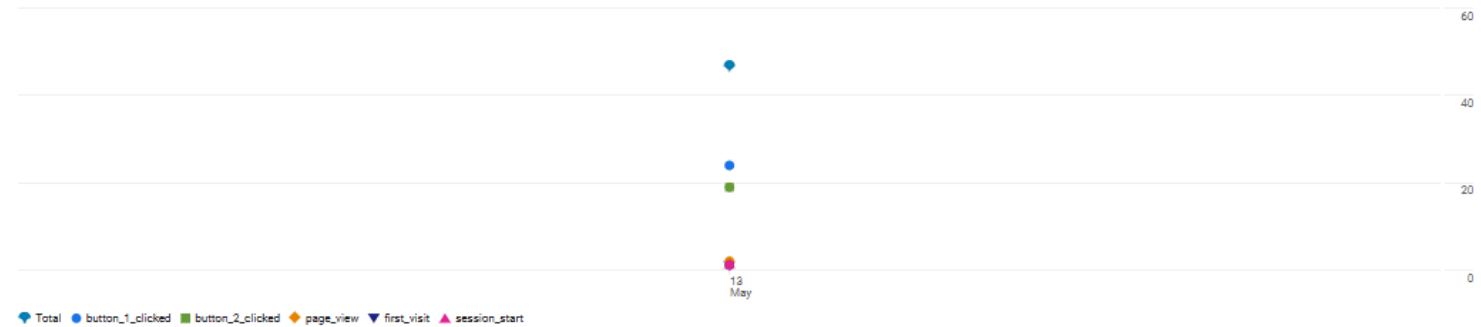
- You can also visit the **Events** section (Analytics->Events) to see more details on the button clicks and other events that are automatically sent to Google Analytics. **Note that it might take some time for the events to show up in the console. Google does not send events in real time.** Read more about [realtime](#) and [debug reports](#) for more frequent and up-to-date information.

Events: Event name

Custom May 13 - May 13, 2026

Event count by Event name over time

Day ▾



Plot rows		Search...		Rows per page: 10 ▾ 1-5 of 5		
	Event name +	↓ Event count	Total users	Event count per active user	Total revenue	
☒	Total	47 100% of total	1 100% of total	47.00 Avg 0%	\$0.00	
☒	1 button_1_clicked	24 (51.06%)	1 (100%)	24.00	\$0.00 (-)	
☒	2 button_2_clicked	19 (40.43%)	1 (100%)	19.00	\$0.00 (-)	
☒	3 page_view	2 (4.26%)	1 (100%)	2.00	\$0.00 (-)	
☒	4 first_visit	1 (2.13%)	1 (100%)	1.00	\$0.00 (-)	

Common pitfalls and best practices

1. Avoid overloading your app with too many tracking events:

- Focus on tracking only the key metrics that directly impact your app's performance and goals.

2. Respect user privacy:

- Always provide transparency on how data is collected and offer users the option to opt out of tracking.

3. Regularly review your analytics reports:

- Set a schedule to review your analytics data to stay updated on trends and respond proactively.

Conclusion and next steps

Here are some things you can explore to further your understanding of Google Analytics and Firebase:

- Try logging different types of user interactions as custom events in Google Analytics. For example, track how users interact with specific features or content within your app.
- Leverage analytics to perform A/B testing on different interfaces or features to see which versions perform better in terms of user engagement and retention.
- Develop more advanced real-time monitoring dashboards in Google Analytics to watch user behavior and app performance live as changes or updates are made to the app.
- Utilize the Google Analytics API to automate the extraction of analytics data and integrate it into custom dashboards or internal reporting tools for more comprehensive analysis.

Congratulations on completing this lab! You have learned how to set up Google Analytics in your Flutter app using Firebase, track key metrics, and use data-driven decisions to improve your app's performance. By understanding user behavior and conversion rates, you can optimize your app to better meet user needs.

Author(s)

Skill-Up EdTech

Lab: Using Google Analytics in Flutter Apps

Estimated time: **30 minutes**



Welcome to this hands-on lab where you will learn how to integrate Google Analytics into your Flutter app using Firebase Analytics. You will explore key features like tracking user interactions, monitoring real-time data, and analyzing user behavior to make data-driven decisions that enhance your app's performance.

Objectives

After completing this lab, you will be able to:

- Set up a basic Flutter application
- Install and configure Firebase Analytics
- Instrument your app to log different types of events

- View and analyze these events in the Firebase console

About Cloud IDE →